requires applying the ConfigMap, Deployment, and Service manifests to the cluster. When using Kustomize, the first step is composing related manifests together so that we can handle them as a single unit. Kustomize does that via a `Kustomization` resource. In the end, we want to let Kustomize manage, process, and generate Kubernetes manifests for us.

Let's see how it works. Open your Catalog Service project (catalog-service) and create a kustomization.yml file inside the k8s folder. It will be the entry point for Kustomize.

We'll first instruct Kustomize about which Kubernetes manifests it should use as a foundation for future customizations. For now, we'll use the existing Deployment and Service manifests.

Listing 14.4 Defining the base Kubernetes manifests for Kustomize

```
apiVersion: kustomize.config.k8s.io/v1beta1     ❶
kind: Kustomization                             ❷

resources:                                      ❸
  - deployment.yml
  - service.yml
```

❶ The API version for Kustomize

❷ The kind of resource defined by the manifest

❸ Kubernetes manifests that Kustomize should manage and process

You might be wondering why we didn't include the ConfigMap. I'm glad you asked! We could have included the configmap.yml file we created earlier in the chapter, but Kustomize offers a better way. Instead of referencing a ConfigMap directly, we can provide a property file and let Kustomize use it to generate a ConfigMap. Let's see how it works.

For starters, let's move the body of the ConfigMap we created previously (configmap.yml) to a new application.yml file within the k8s folder.